ENGENHARIA DE SOFTWARE & SPEC TÉCNICA

Arquitetura de Aplicações Backend em Java

O padrão arquitetural em camadas (Layered Architecture) para Java e Spring Framework fundamenta-se no princípio da Separação de Conceitos (Separation of Concerns). O objetivo central é desacoplar a camada de apresentação HTTP das regras de negócio operacionais e dos detalhes de infraestrutura e persistência de dados, garantindo alta manutenibilidade e facilidade de testes unitários.

1. CAMADAS PRINCIPAIS DO DOMÍNIO E APLICAÇÃO

1.1. Entity / Model (Camada de Persistência e Domínio)

A **Entity** representa o modelo de dados relacional espelhado no código Java via Object-Relational Mapping (ORM), tipicamente gerenciado pelo Jakarta Persistence (JPA) / Hibernate. Elas contêm os atributos da tabela, mapeamentos de chaves primárias (`@Id`), estratégias de geração de sequência, chaves estrangeiras e enums. As entidades devem representar exclusivamente o estado persistente do sistema.

```
@Entity @Table(name = "tb_usuario") public class UsuarioEntity { @Id
@GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; @Column(nullable = false,
unique = true) private String email; @Enumerated(EnumType.STRING) private StatusUsuario status;
// Getters e Setters... }
```

1.2. Resource / Controller (Camada de Exposição REST)

Atua como o ponto de entrada de requisições externas na aplicação. O **Controller** (ou *Resource* na nomenclatura JAX-RS) expõe os endpoints HTTP (GET, POST, PUT, DELETE), mapeia os caminhos da URIs, trata parâmetros de requisição e coordena a desserialização de payloads. Suas responsabilidades devem limitar-se a validar sintaticamente a requisição e invocar o serviço correspondente.

```
@RestController @RequestMapping("/api/v1/usuarios") public class UsuarioController {
private final UsuarioService usuarioService; @PostMapping public
ResponseEntity<UsuarioResponseDTO> criar(@Valid @RequestBody UsuarioForm form) {
UsuarioResponseDTO response = usuarioService.cadastrarUsuario(form); return
ResponseEntity.created(URI.create("/" + response.id())).body(response); } }
```

1.3. Service (Camada de Regras de Negócio)

Contém a inteligência da aplicação. A camada **Service** executa validações operacionais, regras de domínio, orquestração de chamadas entre múltiplos repositórios ou serviços externos e controle transacional (`@Transactional`). Ela isola o comportamento do sistema para que este não dependa da forma como os dados foram recebidos (REST, fila, CLI) ou como são salvos.

```
@Service public class UsuarioService { @Transactional public UsuarioResponseDTO
cadastrarUsuario(UsuarioForm form) { if (usuarioRepository.existsByEmail(form.email())) { throw
new RegraDeNegocioException("E-mail já cadastrado no sistema."); } UsuarioEntity entity =
usuarioMapper.toEntity(form); return usuarioMapper.toDTO(usuarioRepository.save(entity)); } }
```

1. CAMADAS PRINCIPAIS (CONTINUAÇÃO)

1.4. Repository (Camada de Acesso a Dados - DAO)

Abstrai o mecanismo de armazenamento de dados utilizando o padrão *Repository*. Em ecossistemas Spring Data JPA, são interfaces que estendem `JpaRepository<T, ID>`, fornecendo métodos prontos de CRUD (save, findById, delete) e permitindo a declaração de consultas customizadas através de derivados de métodos (ex: `findByEmail`) ou queries explícitas em JPQL/SQL nativo.

```
@Repository public interface UsuarioRepository extends JpaRepository<UsuarioEntity, Long> { boolean existsByEmail(String email); Optional<UsuarioEntity> findByEmailAndStatus(String email, StatusUsuario status); }
```

1.5. Mapper (Camada de Mapeamento de Objetos)

Componente especializado na tradução bidirecional entre diferentes modelos de dados (ex: converter `UsuarioForm` em `UsuarioEntity` e `UsuarioEntity` em `UsuarioResponseDTO`). Ferramentas como o **MapStruct** geram o código de mapeamento em tempo de compilação, eliminando código repetitivo manual e evitando acoplamento entre a entidade de banco e os DTOs externos.

```
@Mapper(componentModel = "spring") public interface UsuarioMapper { UsuarioEntity toEntity(UsuarioForm form); UsuarioResponseDTO toDTO(UsuarioEntity entity); }
```

1.6. DTO e Form (Data Transfer Objects)

Objetos utilitários projetados para trafegar dados entre processos. **Form (Request DTO):** Estrutura de dados de entrada enviada pelo cliente, contendo anotações de validação de campo (ex: `@NotNull`, `@Email`). **DTO (Response DTO):** Estrutura de saída que seleciona apenas os campos que devem ser expostos pela API, omitindo dados sensíveis como senhas ou metadados de banco.

```
public record UsuarioForm( @NotBlank @Email String email, @NotBlank @Size(min = 8) String senha ) {} public record UsuarioResponseDTO(Long id, String email, StatusUsuario status) {}
```

2. MATRIZ DE RESPONSABILIDADE ENTRE CAMADAS

Camada	O que PODE fazer	O que NÃO PODE fazer
Controller	Validar sintaxe, definir rotas HTTP, chamar Services.	Acessar banco diretamente, conter regras de negócio.
Service	Orquestrar regras, controlar transações, chamar Mappers/Repos.	Receber HttpServletRequest, retornar entidades diretas no endpoint.
Repository	Executar queries, gerenciar persistência e SQL.	Tomar decisões operacionais ou aplicar regras de negócio.
Entity	Mapear tabelas, colunas, relacionamentos JPA.	Ser exposta diretamente na interface REST da aplicação.

3. MÓDULOS DE CONFIGURAÇÃO E SUPORTE TÉCNICO

3.1. Config (Configurações Gerais do Framework)

Centraliza as classes anotadas com `@Configuration`, responsáveis por registrar Beans gerenciados pelo IoC Container do Spring. Inclui políticas de segurança web (CORS, CSRF, Spring Security Filter Chains), provedores de autenticação (OAuth2/Keycloak), configurações de cache, instanciacão de clientes HTTP e parâmetros globais de conexão com banco de dados.

```
@Configuration @EnableWebSecurity public class SecurityConfig { @Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { return http.csrf(csrf -> csrf.disable()) .authorizeHttpRequests(auth -> auth.anyRequest().authenticated()) .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults())) .build(); } }
```

3.2. Client (Integrações e Comunicação Externa)

Encapsula chamadas HTTP síncronas/assíncronas para serviços terceiros e microsserviços externos. Utilizando abstrações como **Spring Cloud OpenFeign** ou **WebClient**, o pacote *Client* atua como uma barreira que isola os detalhes do protocolo externo (URLs, Headers de Autenticação, Tokens JWT) da lógica de negócios interna.

```
@FeignClient(name = "keycloak-client", url = "${keycloak.auth-server-url}") public interface KeycloakAdminClient { @PostMapping("/admin/realms/{realm}/users") void criarUsuarioKeycloak(@PathVariable("realm") String realm, @RequestBody KeycloakUserDTO user); }
```

3.3. Exception (Tratamento Global de Erros)

Estrutura que implementa o padrão *Global Exception Handling* através da anotação `@RestControllerAdvice`. Captura exceções lançadas em qualquer ponto da aplicação (ex: `EntityNotFoundException` ou falhas de validação) e as converte em respostas HTTP padronizadas (ex: RFC 7807 Problem Details), impedindo que rastros de pilha (stacktraces) sejam expostos ao cliente.

```
@RestControllerAdvice public class GlobalExceptionHandler { @ExceptionHandler(RegraDeNegocioException.class) public ResponseEntity<ProblemDetail> handleNegocio(RegraDeNegocioException ex) { ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.UNPROCESSABLE_ENTITY, ex.getMessage()); return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(problem); } }
```

3.4. Constant (Valores Fixos e Reutilizáveis)

Pacote destinado ao armazenamento de classes utilitárias de constantes (atributos `public static final` ou *Enums* fechados). Evita a duplicação de literais de texto (*magic strings*) no código-fonte, padronizando mensagens de erro, formatos de data/hora, chaves de cabeçalho HTTP e identificadores do sistema.

3.5. Util (Utilitários Puros e Stateless)

Agrupa métodos estáticos e puros que realizam cálculos, formatações de dados (ex: aplicar máscara em CPF/CNPJ, validação de dígitos verificadores) e manipulações genéricas de listas ou strings. Por definição, classes em `Util` não possuem estado interno e não devem ser injetadas como componentes gerenciados.